

torch.nn.functional.embedding#

<https://pytorch.org/docs/stable/generated/torch.nn.functional.embedding.html>

Description:

Generate a simple lookup table that looks up embeddings in a fixed dictionary and size. This module is often used to retrieve word embeddings using indices. The input to the module is a list of indices, and the embedding matrix, and the output is the corresponding word embeddings. See `torch.nn.Embedding` for more details. Note that the analytical gradients of this function with respect to entries in `weight` at the row specified by `padding_idx` are expected to differ from the numerical ones.

Function Signature

```
torch.nn.functional.embedding(input, weight,
padding_idx=None, max_norm=None, norm_type=2.0,
scale_grad_by_freq=False, sparse=False)[source]#
```

Parameters

Return type

Shape:

Returns:

Tensor

Code Examples

```
>>> # a batch of 2 samples of 4 indices each
>>> input = torch.tensor([[1, 2, 4, 5], [4, 3, 2, 9]])
>>> # an embedding matrix containing 10 tensors of size 3
>>> embedding_matrix = torch.rand(10, 3)
>>> F.embedding(input, embedding_matrix)
tensor([[[ 0.8490,  0.9625,  0.6753],
          [ 0.9666,  0.7761,  0.6108],
          [ 0.6246,  0.9751,  0.3618],
          [ 0.4161,  0.2419,  0.7383]],
        [[ 0.6246,  0.9751,  0.3618],
          [ 0.0237,  0.7794,  0.0528],
          [ 0.9666,  0.7761,  0.6108],
          [ 0.3385,  0.8612,  0.1867]]])

>>> # example with padding_idx
>>> weights = torch.rand(10, 3)
>>> weights[0, :].zero_()
>>> embedding_matrix = weights
>>> input = torch.tensor([[0, 2, 0, 5]])
>>> F.embedding(input, embedding_matrix, padding_idx=0)
tensor([[[ 0.0000,  0.0000,  0.0000],
          [ 0.5609,  0.5384,  0.8720],
          [ 0.0000,  0.0000,  0.0000],
          [ 0.6262,  0.2438,  0.7471]]])
```

Notes & Warnings

NOTE

Note Note that the analytical gradients of this function with respect to entries in weight at the row specified by `padding_idx` are expected to differ from the numerical ones.

NOTE

Note Note that `:class:`torch.nn.Embedding`` differs from this function in that it initializes the row of weight specified by `padding_idx` to all zeros on construction.

Detailed Documentation

Documentation

Generate a simple lookup table that looks up embeddings in a fixed dictionary and size.

This module is often used to retrieve word embeddings using indices. The input to the module is a list of indices, and the embedding matrix, and the output is the corresponding word embeddings.

See `torch.nn.Embedding` for more details.

Note that the analytical gradients of this function with respect to entries in weight at the row specified by `padding_idx` are expected to differ from the numerical ones.

Note that `:class:`torch.nn.Embedding`` differs from this function in that it initializes the

row of weight specified by `padding_idx` to all zeros on construction.

`input` (LongTensor) – Tensor containing indices into the embedding matrix

`weight` (Tensor) – The embedding matrix with number of rows equal to the maximum possible index + 1, and number of columns equal to the embedding size

`padding_idx` (int, optional) – If specified, the entries at `padding_idx` do not contribute to the gradient; therefore, the embedding vector at `padding_idx` is not updated during training, i.e. it remains as a fixed “pad”.

`max_norm` (float, optional) – If given, each embedding vector with norm larger than `max_norm` is renormalized to have norm `max_norm`. Note: this will modify weight in-place.

`norm_type` (float, optional) – The p of the p -norm to compute for the `max_norm` option. Default 2.

`scale_grad_by_freq` (bool, optional) – If given, this will scale gradients by the inverse of frequency of the words in the mini-batch. Default False.

`sparse` (bool, optional) – If True, gradient w.r.t. weight will be a sparse tensor. See Notes under `torch.nn.Embedding` for more details regarding sparse gradients.

Input: LongTensor of arbitrary shape containing the indices to extract

Weight: Embedding matrix of floating point type with shape $(V, \text{embedding_dim})$, where $V = \text{maximum index} + 1$ and `embedding_dim` = the embedding size

Output: $(*, \text{embedding_dim})$, where $*$ is the input shape
